AI Pair Programmer: Code, Debug & Review

Tested AI prompt pack — Word + PDF

AI Pair Programmer: Code, Debug & Review

AI writes code fast and confidently wrong. These prompts force the context, constraints and verification that turn 'fast' into 'safe'. Always review what it ships - that's the job.

Prompt count: 10 · Category: dev · Price: GH₵ 145

Code generation with full context

WHEN TO USE
Writing a non-trivial function.

THE PROMPT

```
Act as a senior [LANGUAGE] developer. Write a function to [TASK]. Context: [PASTE: language version, framework, existing function signatures, coding style]. Constraints: (1) handle edge cases [EDGE CASES], (2) no external dependencies beyond [ALLOWED], (3) follow [STYLE GUIDE], (4) include type hints/docstrings. First write 3 test cases with expected outputs, then the implementation, then any assumptions you made.
```

CUSTOMIZE
Paste real signatures - vague context gives vague code.

WHAT GOOD LOOKS LIKE
Implementation + tests + assumptions you can actually review.

Debug with error + code

WHEN TO USE
You have a failing stack trace.

THE PROMPT

```
Here is my code: [PASTE CODE]. Here is the error: [PASTE ERROR/STACK TRACE]. Here's what I expected to happen: [EXPECTED]. Diagnose in this order: (1) root cause in one sentence, (2) why it happens (the mechanism), (3) the fix with minimal changes, (4) a test that would have caught it. If multiple causes are possible, rank them. Do NOT rewrite the whole file.
```

CUSTOMIZE
Include the line numbers from the trace - it cuts the search space.

WHAT GOOD LOOKS LIKE
A diagnosis you understand, not a mystery patch.

Code review pass

WHEN TO USE
Before merging.

THE PROMPT

```
Review this code like a senior engineer doing a merge review: [PASTE CODE]. Report: (1) bugs (with the failing scenario), (2) security issues, (3) performance problems with complexity notes, (4) readability/maintainability, (5) missing tests. For each finding: severity (critical/major/minor), the fix, and whether it blocks merge. Be specific - quote the lines.
```

CUSTOMIZE
Run it on your own code before you claim it's done.

WHAT GOOD LOOKS LIKE
A review that catches what your rubber duck missed.

SQL query from schema

WHEN TO USE
Writing a business query.

THE PROMPT

```
Act as a senior analytics engineer in [DATABASE - e.g. Postgres]. Here are my tables with columns: [PASTE SCHEMA]. Business question: [PASTE QUESTION]. Write the SQL using: named CTEs with comments, no SELECT *, explicit joins, and a final SELECT with no more than 8 columns. List any assumptions about the data. Then give me a quick way to sanity-check the numbers.
```

CUSTOMIZE
Include row counts if you know them - it changes the approach.

WHAT GOOD LOOKS LIKE
Query + assumptions + a verification step.

Optimize a slow query

WHEN TO USE
Query runs forever.

THE PROMPT

```
Here is my query: [PASTE]. It runs in [X] seconds on ~[N] million rows. Find: (1) top 3 bottlenecks with line references, (2) a rewritten version with each optimization annotated, (3) expected improvement per change, (4) index recommendations (exact DDL), (5) a pre-aggregation strategy if it's still slow. Don't change the result set.
```

CUSTOMIZE
Run EXPLAIN ANALYZE first and paste it - huge signal.

WHAT GOOD LOOKS LIKE
A query plan you can implement and measure.

Write tests first (TDD)

WHEN TO USE
New feature, discipline required.

THE PROMPT

```
I'm building [FEATURE] in [LANGUAGE/FRAMEWORK]. Write the test suite FIRST covering: (1) happy path, (2) edge cases [LIST], (3) failure modes (bad input, missing data, timeouts), (4) one performance sanity test if relevant. Use [TEST FRAMEWORK] conventions. I'll write the implementation to pass these tests. No skipping - if a case is impossible, say why.
```

CUSTOMIZE
TDD with AI works: it turns 'trust me' into 'prove it'.

WHAT GOOD LOOKS LIKE
Tests that define done before a line of implementation.

Explain a codebase fast

WHEN TO USE
Onboarding into unfamiliar code.

THE PROMPT

```
Here is the entry point and file tree of an unfamiliar codebase: [PASTE: main file, key files, package manifest]. Explain: (1) what this app does in 3 sentences, (2) the request lifecycle from entry to response, (3) the 5 most important files and why, (4) where the business logic lives vs. plumbing, (5) the most likely places bugs hide. Assume I know [LANGUAGE] but not this codebase.
```

CUSTOMIZE
Start with the smallest file and grow outward.

WHAT GOOD LOOKS LIKE
A map that turns 'where do I start' into a plan.

Security review of a snippet

WHEN TO USE
Before shipping anything user-facing.

THE PROMPT

```
Security review this code: [PASTE]. Check specifically for: injection (SQL/command/HTML), broken auth or authorization, unsafe deserialization, secret leakage, path traversal, rate-limit gaps, and dependency red flags. For each finding: severity, exploit scenario in one sentence, and the fix. Assume worst-case user input reaches this code.
```

CUSTOMIZE
Run this on your auth and payment code first.

WHAT GOOD LOOKS LIKE
A security pass that finds holes before users do.

Refactor without changing behavior

WHEN TO USE
Code works but is hard to read.

THE PROMPT

```
Refactor this code for readability without changing behavior: [PASTE CODE]. Goals: (1) smaller functions with clear names, (2) remove duplication, (3) early returns over nested conditionals, (4) keep the public interface identical. Show a diff-style before/after for each change and note any place where behavior could accidentally change so I can test it.
```

CUSTOMIZE
Run your tests before and after - behavior is sacred.

WHAT GOOD LOOKS LIKE
Cleaner code with the blast radius mapped.

Deployment checklist for a release

WHEN TO USE
About to ship.

THE PROMPT

```
I'm about to deploy [FEATURE/CHANGE] to [ENV - staging/prod]. Here's what changed: [SUMMARY + DIFF HIGHLIGHTS]. Build a deployment checklist: (1) pre-deploy checks (migrations, config, backups), (2) the deploy steps in order, (3) post-deploy verification (what to check in logs/metrics), (4) rollback plan with the exact revert steps, (5) a 'canary' smoke test script if applicable. Keep it executable, not theoretical.
```

CUSTOMIZE

The rollback plan matters more than the deploy steps.

WHAT GOOD LOOKS LIKE

A checklist that makes releases boring again.